1️⃣ What is a Promise in JavaScript?

A **Promise** in JavaScript is an object that represents the **eventual completion (or failure)** of an asynchronous operation and its resulting value.

In simple terms:

A Promise is a placeholder for a value that will be available **now, later, or never**.

It helps you handle **asynchronous operations** like:

- API calls
- Database requests
- File reading
- Timers
- Authentication requests

2️⃣ Why Do We Need Promises?

Before Promises, we used **callbacks**.

❌ Callback Problem (Callback Hell)

```
getData(function(result1) {
  processData(result1, function(result2) {
    saveData(result2, function(result3) {
      console.log(result3);
    });
  });
});
```

Problems:

- Nested structure
- Hard to read
- Hard to debug

- Error handling is messy

👉 Promises solve this using **chaining**.

3️⃣ Promise States (Lifecycle)

A Promise has **3 states**:

State	Meaning
Pending	Initial state (waiting)
Fulfilled	Operation successful
Rejected	Operation failed

4️⃣ Creating a Promise (Basic Syntax)

```
let promise = new Promise(function(resolve, reject) {

  // async operation

  if(success) {
    resolve("Success Result");
  } else {
    reject("Error Occurred");
  }

});
```

Parameters:

- resolve() → called when operation succeeds
- reject() → called when operation fails

5️⃣ Practical Demo 1 – Simple Promise with setTimeout

Example: Simulating Server Response

```
let myPromise = new Promise(function(resolve, reject) {

    console.log("Fetching data...");

    setTimeout(function() {
        let success = true;

        if(success) {
            resolve("Data fetched successfully!");
        } else {
            reject("Failed to fetch data!");
        }

    }, 2000);

});

myPromise
    .then(function(result) {
        console.log(result);
    })
    .catch(function(error) {
        console.log(error);
```

```
});
```

🔎 **Flow:**

1. Promise created → state = Pending
2. After 2 seconds → resolve/reject
3. .then() handles success
4. .catch() handles error

6️⃣ Promise Chaining

You can chain multiple .then() blocks.

```
let orderFood = new Promise(function(resolve) {
  setTimeout(function() {
    resolve("Order received");
  }, 1000);
});

orderFood
  .then(function(result) {
    console.log(result);
    return "Food is being prepared";
  })
  .then(function(result) {
    console.log(result);
    return "Food delivered";
  })
  .then(function(result) {
    console.log(result);
```

```
});
```

Output:

Order received

Food is being prepared

Food delivered

👉 Each .then() returns a new Promise.

7️⃣ Practical Demo 2 – Real-world Example (Login Simulation)

```
function loginUser(username, password) {

  return new Promise(function(resolve, reject) {

    setTimeout(function() {

      if(username === "admin" && password === "1234") {
        resolve({ status: "success", user: username });
      } else {
        reject({ status: "error", message: "Invalid credentials" });
      }

    }, 1500);

  });

}
```

```
loginUser("admin", "1234")
  .then(function(response) {
    console.log("Login Success:", response);
  })
  .catch(function(error) {
    console.log("Login Failed:", error);
  });
```

8️⃣ Handling Multiple Promises

8.1 Promise.all()

Runs multiple promises in parallel.

```
let p1 = Promise.resolve("Task 1");
let p2 = Promise.resolve("Task 2");
let p3 = Promise.resolve("Task 3");

Promise.all([p1, p2, p3])
  .then(function(results) {
    console.log(results);
  });
```

Output:

```
["Task 1", "Task 2", "Task 3"]
```

👉 If ANY promise fails → entire Promise.all fails.

8.2 Promise.race()

Returns first completed promise.

```
let fast = new Promise(resolve => setTimeout(() => resolve("Fast"), 1000));
```

```
let slow = new Promise(resolve => setTimeout(() => resolve("Slow"), 3000));


Promise.race([fast, slow])
  .then(result => console.log(result));
```

Output:

Fast

💡 Where to Use Promises?

You should use Promises when:

✅ 1. Calling APIs (Fetch)

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

✅ 2. Database Requests

✅ 3. Authentication

✅ 4. File Upload

✅ 5. Delayed Execution

✅ 6. Any Async Task

---

1 0 Promise vs Async/Await

async/await is built on top of Promises.

Promise Style

```
getData()
  .then(result => console.log(result))
```

```
  .catch(error => console.log(error));
```

Async/Await Style (Cleaner)

```
async function fetchData() {
  try {
    let result = await getData();
    console.log(result);
  } catch(error) {
    console.log(error);
  }
}
```

👉 Internally still uses Promise.

---

1 1️⃣ Real Mini Project Example – Order System

```
function placeOrder() {
  return new Promise((resolve) => {
    setTimeout(() => resolve("Order placed"), 1000);
  });
}

function makePayment() {
  return new Promise((resolve) => {
    setTimeout(() => resolve("Payment successful"), 1000);
  });
}

function deliverOrder() {
```

```
    return new Promise((resolve) => {
        setTimeout(() => resolve("Order delivered"), 1000);
    });
}

placeOrder()
    .then(result => {
        console.log(result);
        return makePayment();
    })
    .then(result => {
        console.log(result);
        return deliverOrder();
    })
    .then(result => {
        console.log(result);
    });
```

---

1 4️⃣ Summary

A Promise:

- Handles asynchronous operations
- Avoids callback hell
- Improves readability
- Provides better error handling
- Is the foundation of async/await